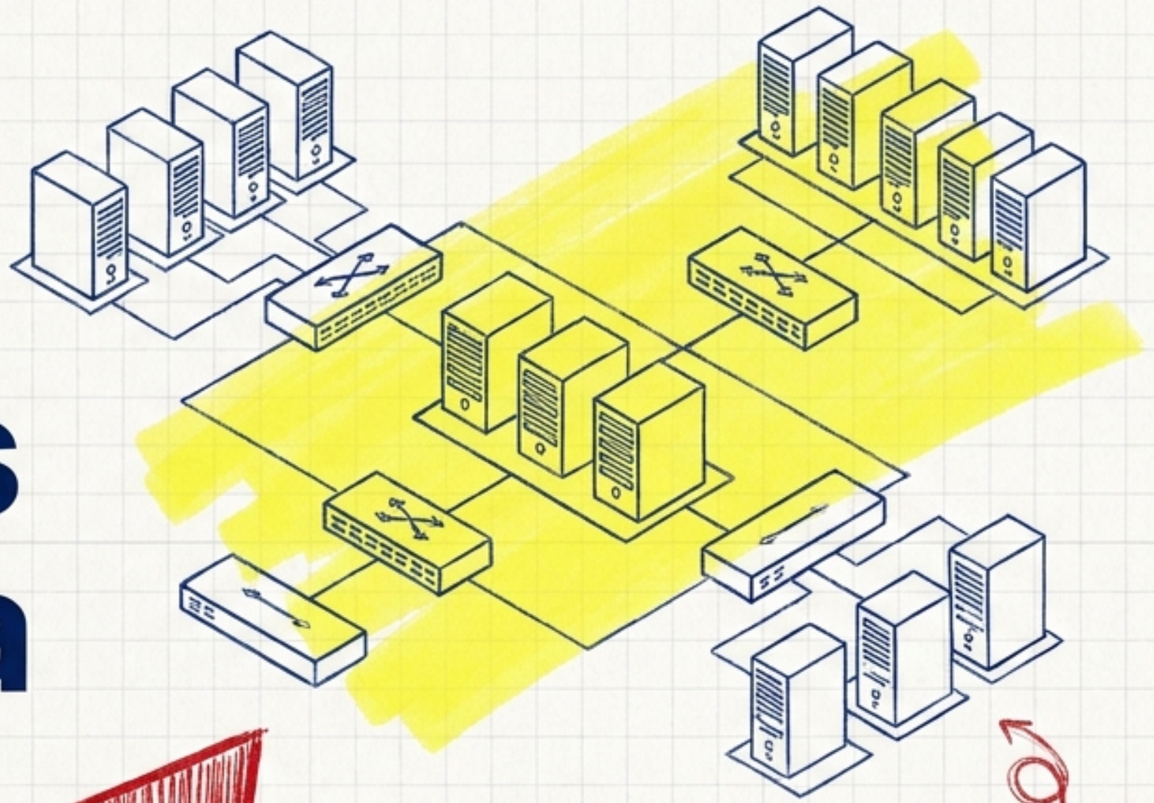
Por Dentro do Código: A Masterclass de Arquitetura



Lições reais sobre concorrência, infraestrutura e pagamentos em sistemas de alta criticidade.

Leia para entender os "porquês", não apenas os "comos".

Gemini Notebook

Qual a única regra que não pode ser quebrada em um marketplace?

[Insight de Entrevista]

Se pedirem o princípio central da sua arquitetura, cite esta frase.

O dinheiro nunca pode ser contado duas vezes nem nem sumir.

Transações Atômicas

Tudo ou nada. Se o estoque falhar, o pagamento nem é registrado.



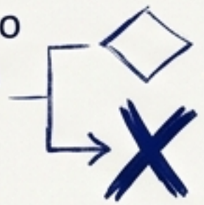
Travas de Linha (select_for_update)

Enfileirar requisições simultâneas direto no banco.



Idempotência

Ignorar o reenvio de cobranças já processadas.



Segurança Fail-Closed

Na dúvida, o sistema recusa a ação.



Gemini Notebook

O Mapa do Território:
Onde mora o MySuperStore?

Internet Pública
(Serverless)

Vercel
(Next.js 15, Zustand, React Query)

Oracle VM Pública
(1GB RAM)

Caddy
(Reverse Proxy/TLS)

Daphne
(ASGI API)

Celery Worker & Redis
(Fila/Canais)

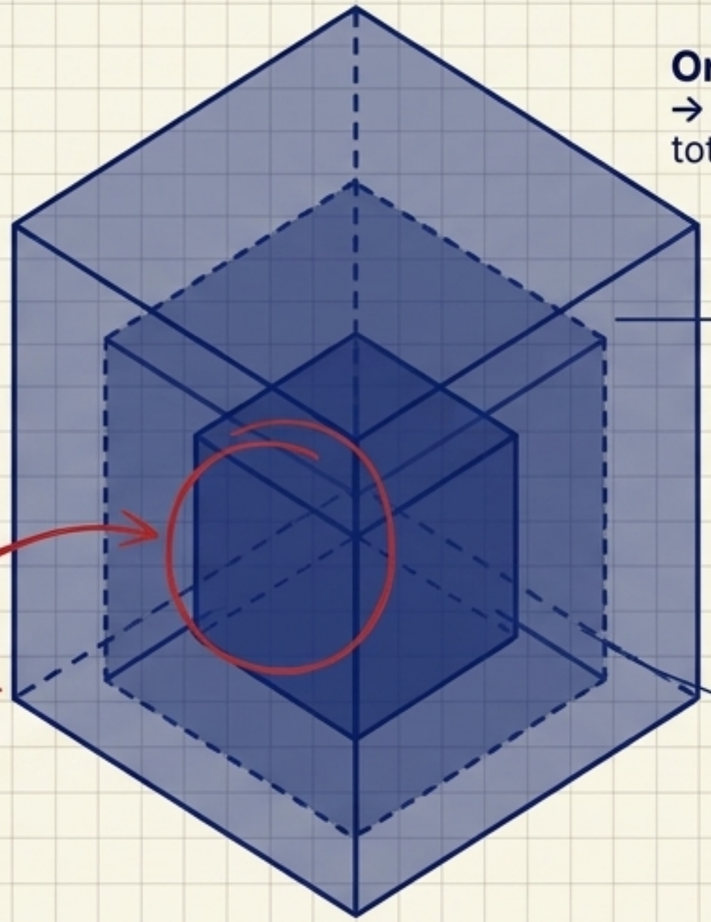
Oracle VM Privada
(Sem Internet)

PostgreSQL 15.
Acessado apenas via túnel SSH.

Por que duas VMs se o banco tem só 11MB?
Isolamento puro. Quem está na internet não tem caminho físico para enxergar o banco de dados.

Gemini Notebook

Por que dividir um pedido em três níveis?



Order (O Cliente)
→ Snapshot do endereço e total consolidado.

SubOrder (A Loja)
→ Dividido por vendedor. Contém o repasse (Payout), comissão da plataforma e frete especifico.

OrderItem (O Produto)
→ Snapshot atômico de nome, SKU e preço com SET_NULL.

O conceito chave é o SNAPSHOT. Copiamos o preço no ato da compra. Se o lojista mudar o preço amanhã, o histórico antigo fica intacto.

Socratic Q&A Panel

Pergunta do Aluno:
Por que não um pedido único?

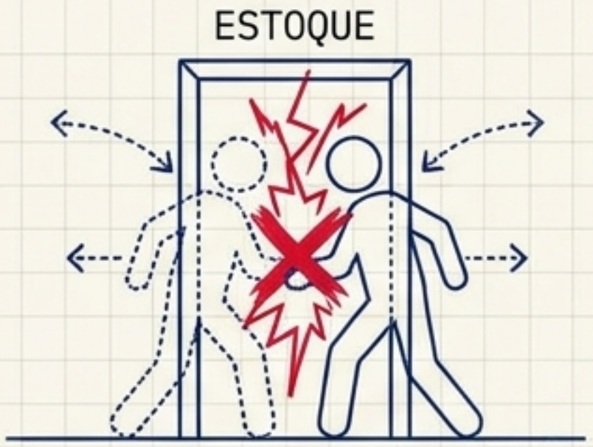
Resposta do Professor:
Porque o cliente compra de 2 lojas num clique, mas cada loja despacha seu pacote, recebe seu dinheiro e tem seu próprio rastreio.

Gemini Notebook

A Corrida Contra o Tempo: Concorrência de Estoque

O Cenário: Duas pessoas tentam comprar a última peça exclusiva (macramê) no exato mesmo milissegundo.

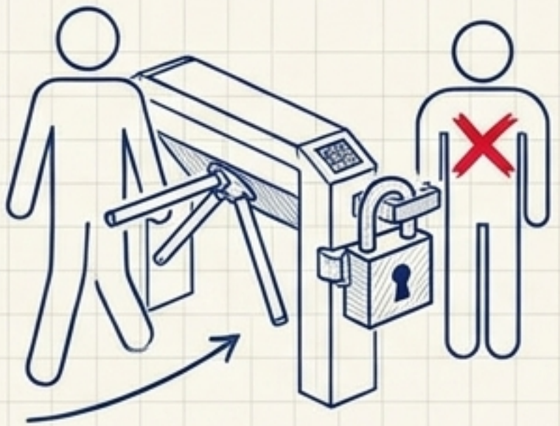
A Solução Clássica (Iniciante / Errada)



Passo: Lê estoque no Python -> Subtrai 1 -> Salva no banco.

Resultado: Ambas as leituras veem "Estoque = 1". Vende-se duas vezes. Gera estoque negativo.

A Solução Sênior (Atômica / Certa)



Código destacado: UPDATE ... WHERE stock >= qtd combinado com select_for_update().

Resultado: A checagem e a subtração acontecem juntas, dentro do banco de dados. Transacional: tudo ou nada.

O banco de dados é a única fonte da verdade.
Tranque a porta do provador enquanto o cliente experimenta a roupa.

Gemini Notebook

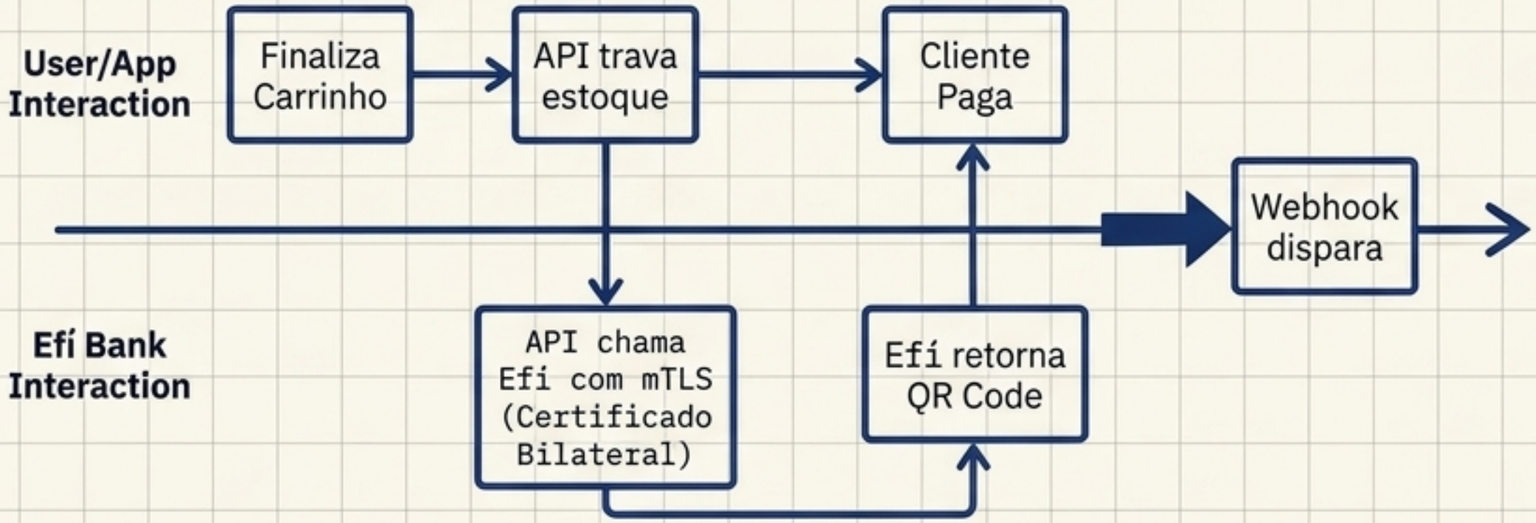
O Caminho do Dinheiro: Como sabemos que o PIX foi pago?

Polling: O passageiro perguntando a cada 5 min "Já chegamos?". (Frontend checando confirm_pix_view).



Webhook: O motorista avisando "Chegamos!". (Efí Bank chamando a API ativamente).





Segurança Fail-Closed

- **Regra**: O webhook é uma URL pública.
- **Defesa**: Exige HMAC (segredo compartilhado). Se a variável de ambiente do segredo faltar no servidor, a API devolve 503 Service Unavailable e recusa TUDO. Nunca confia por padrão.

Gemini Notebook

A Arte da Idempotência: Protegendo o Repasse

O Problema

A rede falha, e o banco reenvia o aviso do mesmo PIX pago. Confirmamos duas vezes? Pagamos duas comissões?

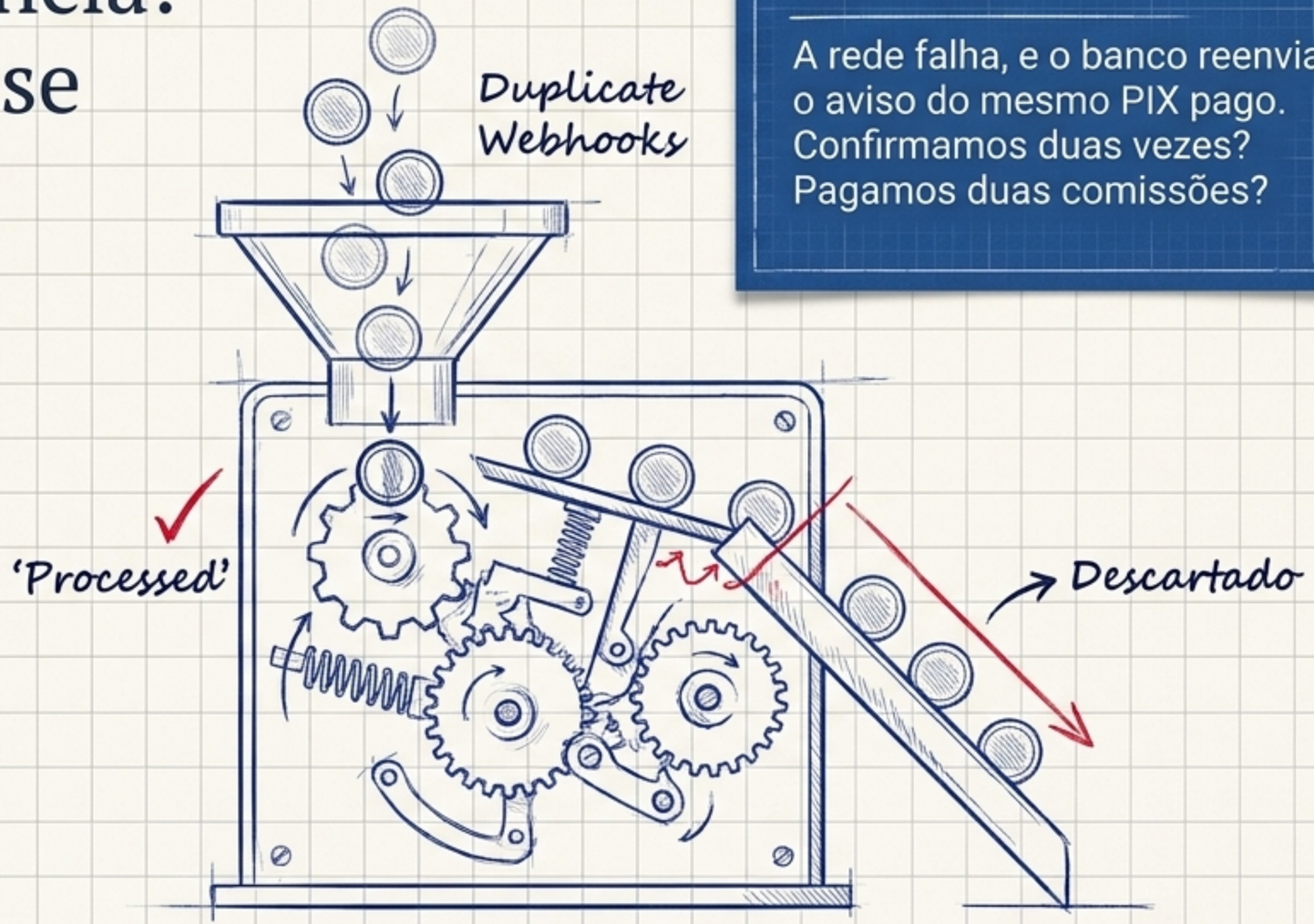
The Solution Logic

A função `process_successful_payment` é acionada:

Passo 1: Trava a linha do pedido no banco.

Passo 2: Checa o status. Já é CONFIRMED?

Passo 3: Se sim, retorna False e morre ali. Nenhuma comissão duplicada.

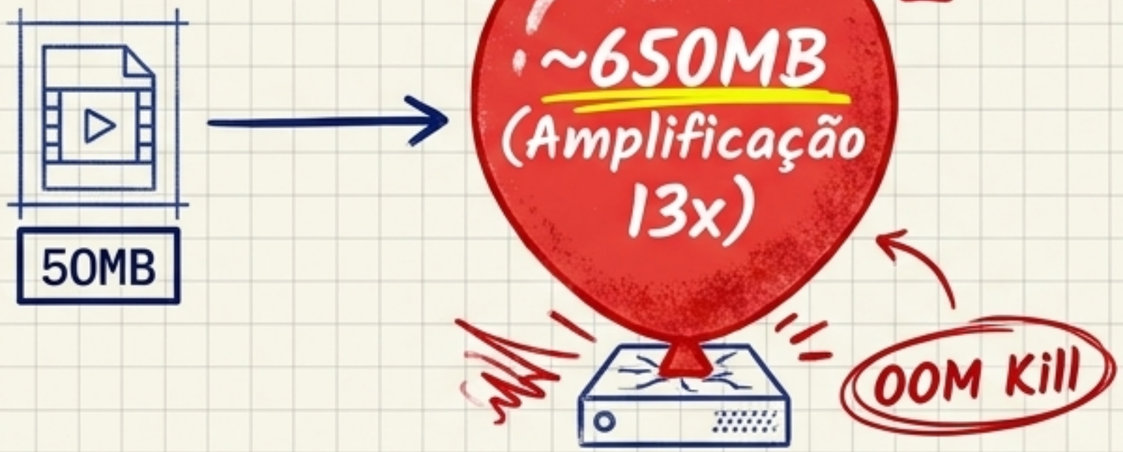


Código idempotente significa que rodar 1 vez ou 100 vezes tem o exato mesmo efeito financeiro da primeira vez.

Gemini Notebook

O Guardião de Memória: Investigando o "OOM Kill"

O Mistério: Em 16/10/2026, como um único upload de vídeo de 50MB matou um servidor inteiro de 1GB de RAM?



A Descoberta (Causa Raiz)

O servidor ASGI (Daphne/Twisted) fazia um "parse duplicado" silencioso de requisições multipart/form-data. Ele lia, decodificava e recopiava os bytes na memória.

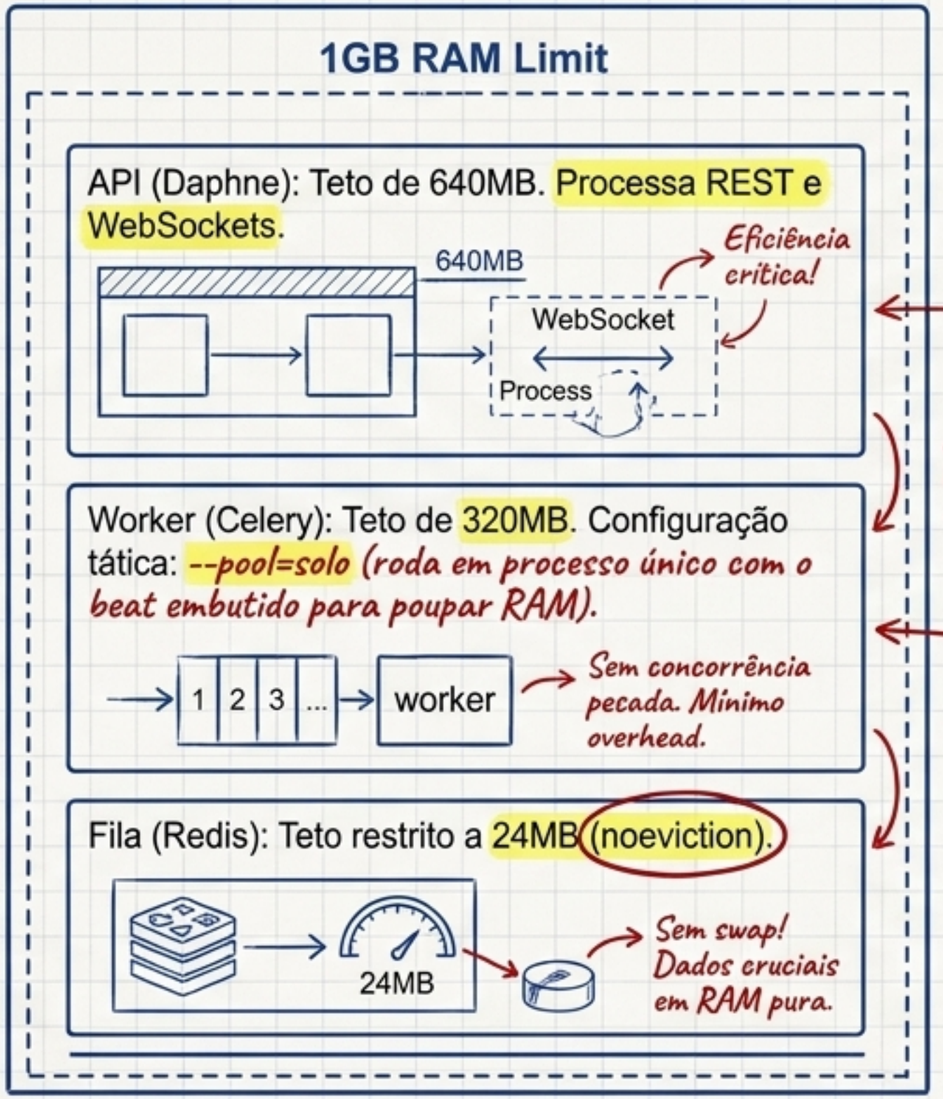
A Resolução Definitiva (Defesa em Profundidade)

1. Desligar o parse duplicado (parsePOSTFormSubmission).
2. Recusar uploads grandes cedo via Content-Length.
3. Upload Direto Assinado: O navegador do cliente envia o vídeo diretamente para o Cloudinary (CDN). A VM nunca mais toca no arquivo pesado.

Gemini Notebook

A Fortaleza Limitada: Operando em 1GB de RAM

Tráfego de Entrada



Survival Tactics

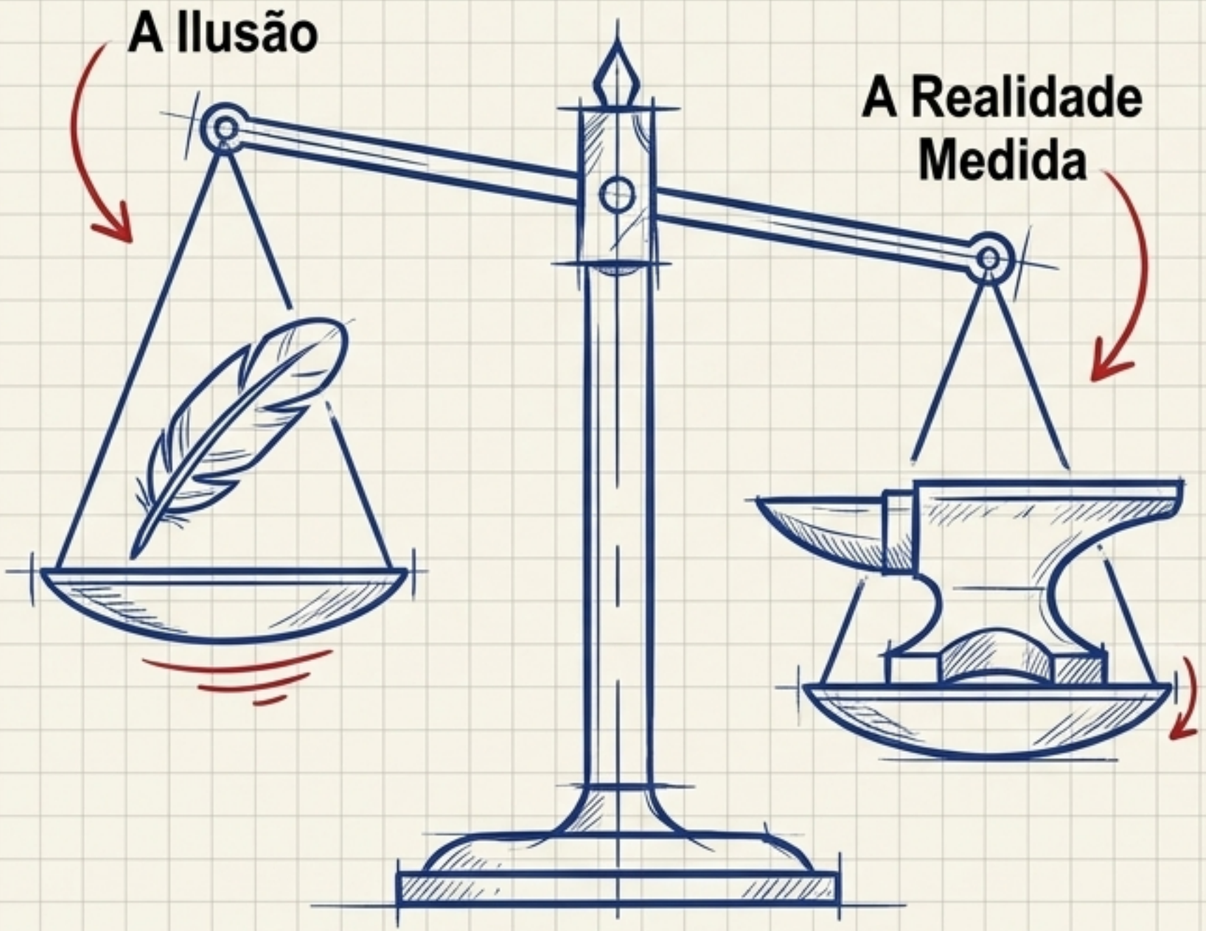
- ☑ Healthchecks ativos: O Docker mata e reinicia contêineres 'zumbis'. *Autocura em ação!*
- ☑ Rotação de Logs: Limitados a 3 arquivos de 5MB (evita disco cheio). *Gerenciamento rigoroso do disco*

Restrições geram excelência. O problema nem sempre é ter pouca memória; às vezes é não saber para onde ela está vazando.

Gemini Notebook

O Paradoxo do Supabase: Medir Antes de Decidir

A Hipótese: “Vamos mover o banco de dados para o Supabase Free. Vai tirar o peso da VM e liberar memória.”



Matriz de Dados

Peso do Banco: Apenas 11MB e 2 conexões.
- A pressão de RAM vinha do Python, não do PostgreSQL.

Latência: Pularia de 1,7 ms (túnel local) para
- ~200 ms (rede externa em Virgínia/EUA).
- Uma tela com 10 queries demoraria 2 segundos só em rede.

A Decisão Sênior

Não migrar.

Uma excelente decisão de arquitetura pode ser "não fazer nada", desde que você tenha os números para provar.

Gemini Notebook

Deploy Sem Medo: CI/CD Defensivo

The Senior's Playbook



Gate 1: Chaves Efêmeras. GitHub Actions usa SSH sem credenciais em disco.

Segurança na fonte!

Gate 2: Run Migrations Primeiro. **(O coração do deploy).**

Gate 3: Healthcheck de Validação. Só roteia o tráfego se o app responder 200 OK.

Validação final, sem downtime.

Pergunta:
Por que rodar as migrações do banco em um contêiner descartável ANTES de subir o novo app?

Resposta:
Se a migração falhar (ex: erro de SQL), o script aborta. A versão antiga do app continua rodando intacta. Protege o uptime.

Zero downtime deployments!

Gemini Notebook

Matriz de Trade-offs: Sistemas em Contraste

The Senior's Playbook

	MercadinhoSys (ERP)	MySuperStore (Marketplace)
Isolamento (Multi-tenant)	Isolamento em código (filtro de linha 'fail-closed'). Barato, mas exige testes rigorosos.	Banco dedicado na VM privada. Seguro nativamente, mas custa mais caro.
Autenticação (Tokens)	Valida o banco a CADA requisição. Demissão bloqueia na hora.	JWT 'refresh' rotativo de 90 dias. Focado em UX de mobile fluido.
Idempotência	Tabela dedicada de Eventos (EfiWebhookEvent).	Checagem direta do Estado do Pedido.

Não existe resposta certa. Existe a resposta adequada ao risco e ao bolso do projeto.

Gemini Notebook

O Raio-X das Vulnerabilidades: Transparência Sênior

The Senior's Playbook

O Raio-X das Vulnerabilidades	
Ameaça	**Fix / Mitigação**
Risco 1: IDOR no WebSocket. Alguém com o UUID entrava no chat. → Falha crítica de acesso!	**Correção:** Validação cruzada de participante no evento 'connect' do canal.
Risco 2: Estoque Preso. Carrinhos abandonados sem escolha de pagamento roubavam inventário.	**Correção:** Cronjob varrendo e limpando pendentes após 60 minutos, devolvendo peças ao pool. → Solução simples e robusta!
Risco 3: Tokens no localStorage. Risco assumido de XSS temporariamente.	**Mitigação Futura:** Transição planejada para cookies HttpOnly com proteção CSRF.

Risco temporário: planejar mitigação ASAP

Esconder dívida técnica é amador. Um engenheiro sênior mapeia as fragilidades e prioriza a correção pelo risco financeiro.

Gemini Notebook

O Pitch do Sênior: Como Vender Suas Decisões

Escalabilidade

Quando perguntarem “Esse sistema de 1GB escala?"

Não diga “Ao infinito”. Diga: “Escalamos por gatilhos. Se o swap do Linux crescer, migramos para uma VM maior. Se exigirem SLA alto em contrato, dividimos em 2 APIs com Load Balancer.”

Segurança

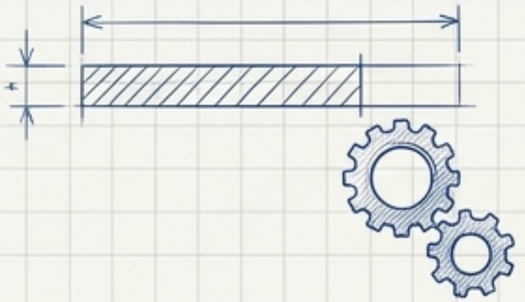
Quando perguntarem "É 100% seguro?"

Não diga “Sim”. Diga: “Usamos defesa em camadas. Temos fail-closed por padrão, mTLS com o banco central, e mapeamos nossos riscos residuais no WebSocket.”

Inventar números destrói sua credibilidade em 2 segundos. “Eu não medi isso ainda, mas mediria assim” é a melhor resposta quando não se sabe.

Gemini Notebook

O Dever de Casa: Prove Que Aprendeu



- **Forçar o Caos:** Escreva um script simulando um Deadlock no banco de dados travando transações em ordem inversa (A->B vs B->A).

Essencial: O script deve ser reprodutível.

- **Quebrar o Isolamento:** Projete um teste de tenant (lojista A tentando ler dados de B) que falhe antes do código ser corrigido.

Prova a falha primeiro!

- **Garantir a Sobrevivência:** Documente e execute um plano real de backup externo (*off-site*), provando a restauração validada de um arquivo .dump.

Validada, não apenas executada.

"Aprender arquitetura não é desenhar caixinhas na lousa. É fazer mudanças pequenas e ter as métricas para provar que funcionaram."

Gemini Notebook